

EARTHQUAKE AND MAGNITUDE PREDICTION USING MACHINE LEARNING

Piotr IMIŃSKI¹, Jan GORZELA², Nadezhda ZHURAVLEVA³

- | | |
|-----------------------|------------------------------|
| 1. Student ID: 247088 | e-mail: 247088@edu.p.lodz.pl |
| 2. Student ID: 247086 | e-mail: 247086@edu.p.lodz.pl |
| 3. Student ID: 248612 | e-mail: 248612@edu.p.lodz.pl |

Abstract: This report explores machine learning approaches to predict earthquake magnitudes using two datasets: Kaggle's global earthquake dataset and SOCR's Northern California dataset. We implemented and evaluated multiple algorithms, including Random Forest, Linear Regression, Naïve Bayes, and Neural Networks, assessing their performance with metrics such as R^2 and Mean Squared Error. The study highlights Random Forest as the most effective algorithm for magnitude prediction, achieving an R^2 of 0.73 due to its capability to handle complex patterns. While challenges such as dataset quality and algorithm performance were encountered, the research provides valuable insights into the potential of machine learning for seismic magnitude prediction.

Keywords: Earthquake prediction, Machine learning, Magnitude estimation, Random Forest, Seismic data analysis.

1. INTRODUCTION

Earthquakes are among the most devastating natural disasters, causing significant loss of life, infrastructure damage, and economic disruption. Predicting earthquake magnitudes is a critical challenge that could improve early warning systems, disaster preparedness, and resource allocation. Despite advances in seismology, accurately forecasting earthquake characteristics remains a complex problem due to the nonlinear and multi-dimensional nature of seismic data.

This project aims to explore the use of machine learning techniques for predicting earthquake magnitudes using two distinct datasets: Kaggle's global earthquake dataset and SOCR's Northern California dataset. Machine learning offers the potential to identify patterns and relationships in data that traditional statistical methods might overlook, making it a promising tool for this application.

The motivation behind this work stems from the increasing availability of seismic data and the growing interest in leveraging machine learning to address complex geophysical problems. By evaluating and comparing various algorithms, this project seeks to identify the most effective model for magnitude prediction and provide insights into the features and techniques that contribute to its success.

Previous research in this field has focused on different methods, such as statistical modelling and neural networks, often limited by small datasets, overfitting, or narrow scope in terms of geographic applicability. Our work builds on these studies by applying diverse algorithms, including Random Forest, Linear Regression, and Neural Networks, and by leveraging both global and region-specific datasets. Additionally, we address common challenges such as feature

selection, data preprocessing, and hyperparameter optimization to improve model performance.

In summary, this project explores how machine learning can enhance earthquake magnitude prediction, evaluates multiple algorithms on different datasets, and highlights the strengths and limitations of the approaches employed.

2. MATERIALS AND METHODS

2.1. Data

The success of machine learning model depends on data quality and preparation. For this project we looked into two publicly available datasets:

- Earthquake dataset on Kaggle [1]
- SOCR Earthquake Data [2]

The Kaggle dataset consists of 19 features and 1000 entries from different places on earth and the data was collected from 1 January 1996 till 31 October 2019. The locations of the entries are presented in Fig. 1.

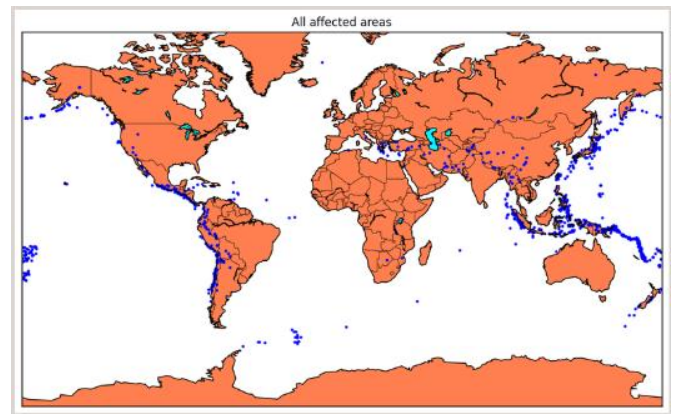


Fig. 1. Visualisation of entries from Kaggle dataset.

What is more the values of magnitude are more diverse than the magnitudes from the second dataset. The minimum value of magnitude is 6.5 and maximum is 9.1. The distribution of magnitude is presented in figure 2. After looking deeper into the data we find out that the Kaggle dataset consists also of more features.

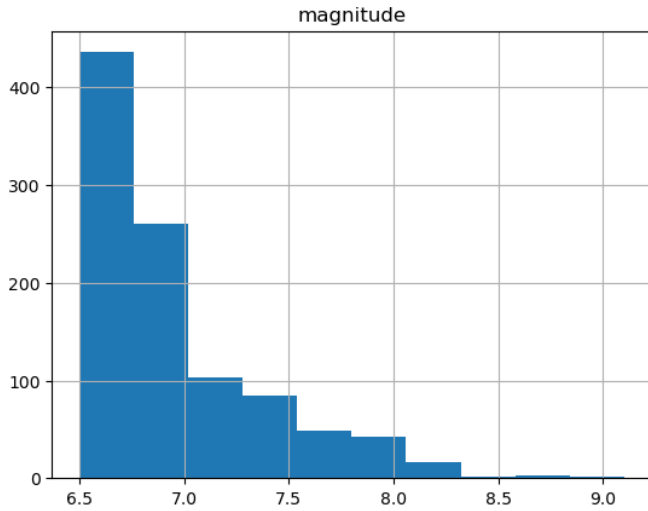


Fig. 2. Distribution of magnitudes in Kaggle dataset.

However, on the other hand the dataset from the Northern California consists of much more data points – 18030 entries, but less features – only 13. What is more all of the entries are from the small region, the dataset consists of entries only from the Northern California. The visualization of the data points is presented in Fig. 3.

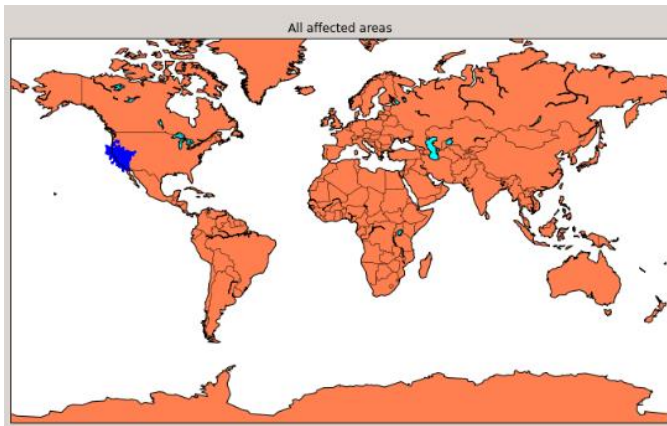


Fig. 3. Visualisation of entries from SOCR dataset.

The data was collected from 1 July 1966 till 28 December 2007. The minimum value is 3.0 and the maximum value is 7.39. The distribution is presented in Fig. 4.

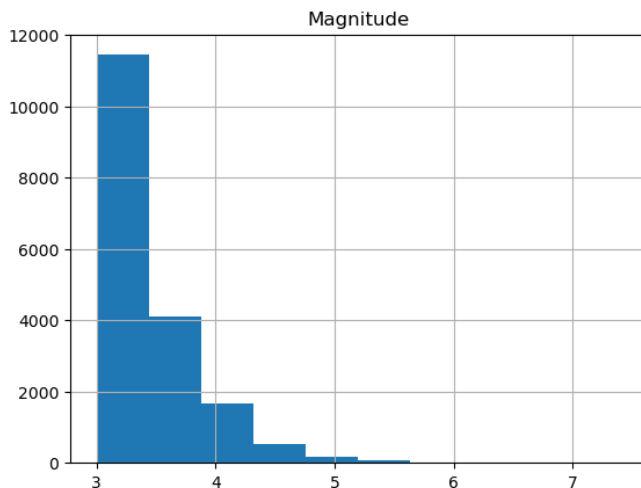


Fig. 4. Distribution of magnitudes in SOCR dataset.

Because of that discrepancies we decided to implement algorithms separately on both datasets and compare their correctness.

After choosing the datasets we proceed into feature selection step. In order to choose the optimal features we normalized the data, rejected the features with too many missing values, and created a correlation matrix in order to look for any correlation between features. The correlation matrix for Kaggle dataset is presented in figure 5, and for the SOCR dataset in figure 6. After that step we were able to choose the features to be used in machine learning algorithms.

	magnitude	cdi	mml	tsunami	sig	nst	dmin	gap	depth	latitude	longitude
magnitude	1.00	0.16	0.28	-0.00	0.51	0.09	-0.08	-0.09	0.02	-0.02	-0.01
cdi	0.16	1.00	0.20	0.29	0.48	-0.00	0.14	0.28	-0.05	0.07	-0.13
mml	0.28	0.20	1.00	-0.18	0.40	0.11	-0.31	-0.07	-0.52	0.18	-0.02
tsunami	-0.00	0.29	-0.18	1.00	0.02	-0.43	0.44	0.24	0.07	-0.13	-0.13
sig	0.51	0.48	0.40	0.02	1.00	0.02	-0.05	0.16	-0.08	0.18	-0.16
nst	0.09	-0.00	0.11	-0.43	0.02	1.00	-0.40	0.00	-0.10	0.11	0.16
dmin	-0.08	0.14	-0.31	0.44	-0.05	-0.40	1.00	0.09	0.19	-0.24	-0.08
gap	-0.09	0.28	-0.07	0.24	0.16	0.00	0.09	1.00	-0.06	0.05	-0.27
depth	0.02	-0.05	-0.52	0.07	-0.08	-0.10	0.19	-0.06	1.00	-0.10	-0.03
latitude	-0.02	0.07	0.18	-0.13	0.18	0.11	-0.24	0.05	-0.10	1.00	0.01
longitude	-0.01	-0.13	-0.02	-0.13	-0.16	0.16	-0.08	-0.27	-0.03	0.01	1.00

Fig. 5. Correlation matrix for features in Kaggle dataset.

	magnitude	cdi	mml	tsunami	sig	nst	dmin	gap	depth	latitude	longitude
magnitude	1.000000	0.163170	0.282275	-0.001213	0.512358	0.091427	-0.081518	-0.087029	0.017083	-0.017505	-0.010710
cdi	0.163170	1.000000	0.197458	0.292202	0.479911	-0.003857	0.140505	0.277689	-0.051331	0.065383	-0.132190
mml	0.282275	0.197458	1.000000	-0.175441	0.398149	0.110260	-0.307331	-0.048503	-0.517002	0.180799	-0.019398
tsunami	-0.001213	0.292202	-0.175441	1.000000	0.034848	-0.426026	0.443179	0.236259	0.070705	-0.127002	-0.131340
sig	0.512358	0.479911	0.398149	0.034848	1.000000	0.016636	-0.051440	0.158853	-0.079263	0.177575	-0.164200
nst	0.091427	-0.003857	0.110260	-0.426026	0.016636	1.000000	-0.402445	0.003023	-0.103897	0.109991	0.156543
dmin	-0.081518	0.140505	-0.307331	0.443179	-0.051440	-0.402445	1.000000	0.091388	0.192698	-0.239791	-0.075595
gap	-0.087029	0.277689	-0.048503	0.236259	0.158853	0.003023	0.091388	1.000000	-0.062215	0.052368	-0.267812
depth	0.017083	-0.051331	-0.517002	0.070705	-0.079263	-0.103897	0.192698	-0.062215	1.000000	-0.101446	-0.030631
latitude	-0.017505	0.065383	0.180799	-0.127002	0.177575	0.109991	-0.239791	0.052368	-0.101446	1.000000	0.013604
longitude	-0.010710	-0.132190	-0.019398	-0.131340	-0.164200	0.156543	0.075595	-0.267812	-0.030631	0.013604	1.000000

Fig. 6. Correlation matrix for SOCR dataset.

2.2. Tools and technologies

The experiments conducted in this project were carried out on a standard hardware setup consisting of a CPU with Windows 11 as the operating system. No specialized hardware, such as GPUs, was utilized for this study, ensuring the methods remain accessible and reproducible on widely available systems.

The primary software environment included Python as the programming language, with Scikit-learn as the main framework for implementing and training machine learning models. It provides simple and efficient tools for data mining and ML, it is built on NumPy, SciPy, and Matplotlib so it is reliable and easy to use.

During implementation of neural network we used Tensor Flow, which is widely used for building and deploying ML and deep learning models, from simple linear regression to complex neural network. It was chosen for its flexibility, ease of use as well as a wide range of examples.

Additionally, several Python libraries were used to facilitate data preprocessing, feature engineering, visualization, and evaluation. These libraries include:

- **NumPy:** For numerical computations.
- **Pandas:** For data manipulation and analysis.

- **Matplotlib** and **mpl_toolkits**: For creating visualizations.
- **Statsmodels**: For Time Series

In Table 1 we presented the versions of used libraries.

Table 1. Versions of used frameworks and libraries.

Name	Version
python	3.12.7
sklearn	1.5.2
tensorflow	2.18.0
statsmodels	0.14.2
numpy	2.0.2
pandas	2.2.3
matplotlib	3.9.2

2.3. Methods and experimental setup

To explore machine learning methods for earthquake detection, we began by using Orange Data Mining, a visual programming platform, to simulate multiple algorithms and analyse their performance. This initial step allowed us to gain insights into the potential of different approaches before implementing them independently.

We conducted simulations using both datasets: the Kaggle dataset and the Northern California dataset. For these experiments, the target feature was magnitude.

To improve the models' performance, we applied Principal Component Analysis (PCA) to reduce the number of features while retaining the most significant information. The results varied depending on the dataset and algorithm:

- For the larger Northern California dataset, all algorithms struggled, producing low performance metrics due to the limited magnitude and location parameters variance.
- For the smaller Kaggle dataset, Random Forest performed significantly better than other algorithms, achieving an R^2 score of 0.77, making it the most promising candidate for further exploration.

To reproduce results from various sources, we employed several algorithms to solve the problem. The algorithms tested included:

- Linear regression [3]
- Naïve Bayes [3]
- Random forest [4]
- Neural network [4]
- Time Series [5]

2.3.1. Linear regression

Linear regression was successfully performed on SOCR database. Our model has the following characteristics:

Input Data:

- Features: Latitude, Longitude, Depth, No stations, and Gap.

Target:

- Magnitude.

Data Splitting:

We have split the data into training (80%) and testing (20%) sets.

The performance of our linear regression model was evaluated using two metrics: $R^2 = 0.06$ and $MSE = 0.01$, indicating a poor fit to the data. This suggests that the model is unable to capture the underlying pattern. A potential improvement could be to use polynomial regression, which can capture nonlinear relationships. However, our goal was to replicate the model described in the source, where only linear regression was specified as the method of choice.

2.3.2. Naïve Bayes.

Naïve Bayes was also performed on SOCR dataset. To evaluate this method magnitude values were categorized into the following groups:

- magnitudes between $[0, 2)$ are classified as 'Micro',
- between $[2, 3)$ as 'Minor',
- between $[3, 4)$ as 'Light',
- between $[4, 5)$ as 'Moderate',
- between $[5, 6)$ as 'Strong',
- between $[6, 7)$ as 'Major',
- and magnitudes of $[7, \infty)$ as 'Great'.

It should be also mentioned that 'Micro' category was highly overrepresented (having 15 370 instances out of 18 030).

Our model has the following characteristics:

Input Data:

- Features: Latitude(deg), Longitude(deg), No_of_Stations.
- Target: Magnitude_Category_Encoded.

Data Splitting:

We have split the data into training (70%) and testing (30%) sets.

If we evaluate our model using accuracy, the achieved score is 0.985, indicating that the model correctly predicted the target variable approximately 98.5% of the time. However, upon examining the confusion matrix (Fig. 7), it becomes clear that the model fails to capture the underlying patterns effectively, as it predominantly predicts the overrepresented category. This suggests that while accuracy is high, the model is not generalizing well and is likely biased toward the majority class.

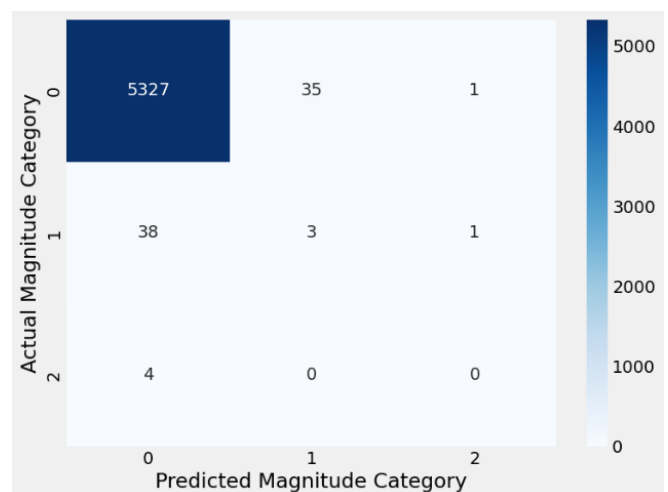


Fig. 7. Confusion matrix.

2.3.3. Random forest (ver 1).

We performed Random Forest on the SOCR database, focusing on using few features trying to recreate findings from

the article [6]. Additionally, we transformed the `date_time` datatype into a timestamp format for future use.

Our model has the following characteristics:

Input Data:

- Features: Latitude, Longitude, Timestamp.
- Target: Magnitude, Depth.

Data Splitting:

We have split the data into training (80%) and testing (20%) sets.

Unfortunately, we were unable to reproduce the performance of the model found in the source. The performance of our random forest model was evaluated using two metrics: $R^2 = 0.11$ (the source has $R^2 = 0.86$) and $MSE = 5953.8$, indicating a poor fit to the data.

2.3.4. Neural networks

Neural networks are well suited to be used in developing machine learning models for complex tasks such as earthquake detection. It is mostly because of their ability to learn intricate patterns from data. So they can be used with seismic data which often exhibits nonlinear relationships between features.

A neural network is a computational model that was inspired by human brain. It consists of layers of interconnected nodes, which are some functions that are processing the data and passing it through different transformations in order to make a prediction. The neural network consists of:

- Input layer: Receives the input features
- Hidden layer: Performs computations, consists of activation functions and is responsible for learning the patterns
- Output layer: Produces the final predictions

In particular, for earthquake detection, a neural network can be used to classify seismic events or predict characteristics such as the magnitude or likelihood of occurrence in a specific region. In our case we tried to reconstruct the approach proposed in article [6], therefore we focused on determining the magnitude of an earthquake.

In order to train the neural network its weights and biases needs to be optimized to minimize the difference between predicted and actual outputs. The key steps includes choosing:

- The loss function
- Optimizer
- Number of batches and Epochs
- Validation set

In our approach we tried several different configurations in order to obtain the best solution.

To train the model we used the SOCR Earthquake dataset. As input we used timestamp, longitude, magnitude and depth. Which were normalized by `MinMaxScaler`. We divided our dataset into train and test set by `train_test_split` from `sklearn` with test size of 0.2.

In order to find the best parameters for the neural network we used Grid Search algorithm. After running it we were able to create neural network with optimal architecture. Our neural network consists of one input layer (4), two dense layers with 64 neurons and ReLu activation function and a dense layer with one neuron on the output, which will return predicted magnitude. As optimizer we used Adam with 0.0001 learning rate and for a loss function we used Mean Squared Error. During the model fitting process we set the bath size to 40 and

number of epochs to 400, however we run the process with early stepping function with patience of 10 epochs.

With such implementation of the neural network we obtained:

- Mean Absolute Error: 0.30
- Mean Squared Error: 0.17
- R^2 : 0.03

Unfortunately the results obtained by using this method weren't very promising and we moved on to explore different algorithms.

2.3.5. Time Series

Another approach to our problem can consists of the use of the time series.

A time series is a sequence of data points representing the successive points in time, typically spaced at uniform intervals. It captures how a variable evolves over time and can also be used in seismology. Time series analysis aims to identify patterns, understand the underlying structure, and make predictions about future values.

In order to try that approach we started by preparing our dataset. We expanded the dataset of the missing dates by inserting them with corresponding magnitude of 0. Then we calculated the mean value of magnitude for every month. After the visualization of the data (Fig. 8) one can see slight seasonality of 6 years.

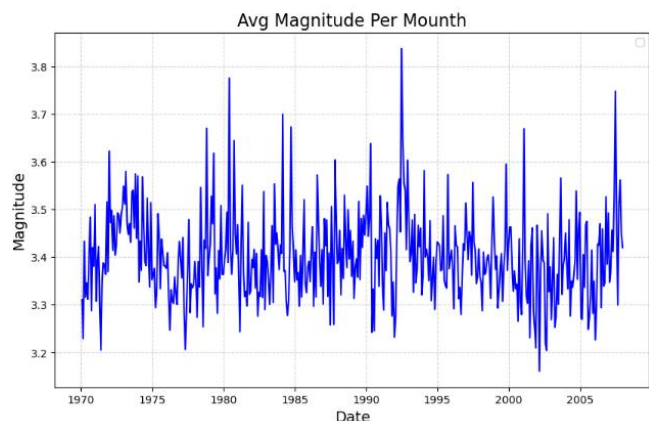


Fig. 8. Visualisation of the magnitudes.

In order to create predictions we used the statistical model SARIMAX (Seasonal AutoRegressive Integrated Moving Average with eXogenous variables). This model is an advanced time series forecasting technique that builds upon the ARIMA framework by incorporating both seasonal patterns and external variables. The SARIMAX model is defined by the following parameters:

- p (AR Order): Number of previous time steps used to predict the next value.
- d (Differencing): Number of times the data needs to be differenced to make it stationary (remove trends).
- q (MA Order): Number of past error terms (mistakes the model made) included to improve predictions.
- P (Seasonal AR Order): Similar to p, but considers seasonal lags.
- D (Seasonal Differencing): Removes seasonal trends by differencing at seasonal intervals.
- Q (Seasonal MA Order): Like q, but for seasonal errors.

- s (Seasonal Period): The length of the seasonal cycle.

Before training the model we divided our data into training and testing set, where the testing set consists of the magnitude values from the last 12 months.

As the input to the model we provided the magnitudes values averaged for every month with date as the indexes. As for the parameters of the model we choose them as follows:

- (p, d, q) = (1, 1, 1)
- (P, D, Q, s) = (1, 1, 1, 72)

The prediction of the model for the test set is presented in Fig. 9.

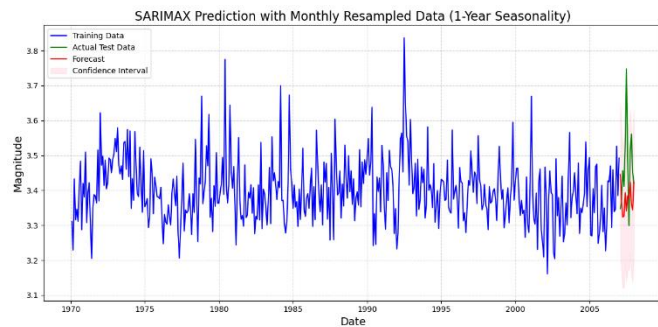


Fig. 9. Prediction of the model.

Despite the fact that the test values are indeed within the confidence interval, the R^2 isn't very optimistic:

- Mean Absolute Error: 0.12
- Mean Squared Error: 0.02
- R^2 : -0.94

Based on that we concluded that, however it probably could be used in such problem one would have to have greater dataset and more computational power.

2.3.6. Random Forest (ver 2)

For the Random Forest implementation, we focused on predicting earthquake magnitudes using a carefully selected set of features from the Kaggle dataset. These features represent various earthquake-related metrics:

- CDI (Community Determined Intensity): Measures the perceived intensity of the earthquake.
- MMI (Modified Mercalli Intensity): A scale to quantify the effects of an earthquake on structures and people.
- Tsunami: A binary indicator showing whether the earthquake triggered a tsunami.
- Sig (Significance): A value estimating the overall significance of the earthquake based on various parameters.
- Nst (Number of Stations): The number of seismic stations that recorded the event.
- Dmin (Minimum Distance): The closest distance between the earthquake's epicentre and a seismic station.
- Gap: The largest azimuthal gap (in degrees) in station coverage.
- Depth: The depth of the earthquake in kilometres.
- Latitude and Longitude: The geographic coordinates of the earthquake's epicentre.

- Timestamp: A numerical representation of the date and time of the earthquake.

These features were chosen for their potential relevance to earthquake magnitude prediction. The timestamp transformation from date-time to a numerical format was particularly crucial for improving the model's understanding of temporal patterns.

To optimize the Random Forest model, we used Grid Search to identify the best combination of hyperparameters. The dataset was split into 80% for training and 20% for testing, and the following optimal hyperparameters were found:

- Number of estimators (n_estimators): 97
- Minimum samples split (min_samples_split): 5
- Minimum samples leaf (min_samples_leaf): 2
- Maximum features (max_features): sqrt
- Maximum depth (max_depth): 40
- Bootstrap: False

With these settings, the Random Forest model achieved the following performance metrics:

- R^2 : 0.73
- Mean Absolute Error (MAE): 0.13
- Mean Squared Error (MSE): 0.5

These results confirm that the **Random Forest model** is the most effective approach explored in this study, thanks to careful feature selection and hyperparameter tuning using Grid Search.

3. RESULTS

This section presents the qualitative and quantitative outcomes of the machine learning algorithms applied to earthquake magnitude prediction. The results are evaluated using metrics such as R^2 , Mean Squared Error (MSE), and accuracy. Additionally, comparisons between the algorithms and their respective datasets are provided to assess their relative performance.

The results of the implemented algorithms are summarized in Table 1. The evaluation focused on the accuracy of predictions, the ability to generalize, and the computational efficiency.

The Random Forest algorithm demonstrated significant improvements over other methods. Its R^2 value of 0.73 indicates its capability to handle complex patterns in the Kaggle dataset.

Table 2. Comparison of different algorithms.

Algorithm	R^2	MSE	INPUT	OUTPUT
Linear regression	0.06	0.01	Latitude, Longitude, Depth, No stations, and Gap	Magnitude

Random forest	0.10	5953.8	Latitude, Longitude, Timestamp	Magnitude, Depth
Neural network	0.03	0.17	Latitude, Longitude, Timestamp	Magnitude
Time Series	-0.94	0.02	magnitudes values averaged for every month with date as the indexes	Average mounthly magnitude for next year.
Random forest	0.73	0.5	CDI, MMI, Tsunami, Sig, Nst, Dmin, Gap, Depth, Latitude, Longitude, Timestamp.	Magnitude
Accuracy				
Naïve Bayes	0.985		Latitude(deg), Longitude(deg), No_of_Stations	Magnitude_Category_Encoded

The Random Forest algorithm demonstrated significant improvements over other methods. Its R^2 value of 0.73 indicates its capability to handle complex patterns in the Kaggle dataset. Other algorithms, such as Neural Networks, and Linear Regression, had significantly worse outcomes compared to the results reported in papers and articles, likely due to dataset limitations, differences in preprocessing techniques, and computational constraints during implementation.

4. DISCUSSION

The results of our analysis reveal considerable variations in the predictive performance of different algorithms, emphasizing the importance of model selection and feature engineering in earthquake magnitude prediction. Among the tested approaches, the Random Forest model incorporating a comprehensive set of features (CDI, MMI, Tsunami, Sig, Nst, Dmin, Gap, Depth, Latitude, Longitude, Timestamp) demonstrated the highest accuracy ($R^2 = 0.73$, $MSE = 0.5$). This result highlights the model's strength in capturing complex relationships and nonlinear dependencies in the data. The success of this approach underscores the crucial role of selecting relevant features, as well as the ability of ensemble methods to handle diverse and intricate patterns effectively. In contrast, other models struggled to achieve meaningful predictive performance. Linear regression ($R^2 = 0.06$, $MSE = 0.01$) failed to establish a strong relationship between input features and earthquake magnitude, suggesting that a simple linear model is insufficient for capturing the complexity of seismic events. Similarly, the neural network model ($R^2 = 0.03$, $MSE = 0.17$) underperformed, despite its theoretical ability to capture nonlinear dependencies. This may be due to insufficient training data, suboptimal hyperparameter tuning, or the lack of key predictive features. The time series model

performed particularly poorly ($R^2 = -0.94$), indicating that the averaged monthly magnitudes did not provide sufficient temporal patterns for accurate long-term forecasting.

An important aspect of our methodology was the reliance on approaches described in scientific papers when implementing the models. Despite following established methods, we were unable to replicate their reported success. This discrepancy highlights potential issues such as differences in dataset characteristics, preprocessing techniques, or parameter settings that were not explicitly detailed in the literature. It also underscores a common challenge in machine learning research—models that perform well in one study may not necessarily generalize to different datasets or contexts. This finding suggests that earthquake prediction remains highly dependent on data-specific factors, and blindly applying methods from prior studies without adaptation may lead to suboptimal results.

Despite the relative success of Random Forest, its performance is not flawless, indicating room for improvement. One limitation is the potential for overfitting due to the large number of features used. Additionally, since the model relies entirely on historical data, it may struggle with unseen seismic events that deviate significantly from past patterns. Future research could explore advanced feature selection techniques, ensemble methods that integrate multiple models, or the inclusion of additional geophysical parameters and different datasets.

In summary, our findings emphasize the importance of dataset and feature selection as well as careful model evaluation in earthquake prediction. While Random Forest demonstrated strong predictive performance, the failure of other models, despite being based on established scientific methodologies, highlights the complexity and dataset dependency of this task. Addressing these limitations through improved data representation, adaptive modelling techniques, and alternative methodologies will be crucial for advancing predictive capabilities in seismology. These results also reinforce the need for a more nuanced approach to machine learning in scientific applications, where model generalization and reproducibility remain significant challenges.

5. CONCLUSIONS

In conclusion, this study set out to evaluate multiple algorithms to determine the most effective one for predicting the magnitude. After carefully preparing and cleaning the data, we implemented various models, including linear regression, Naïve Bayes, neural networks, time series and Random Forest. Through a detailed comparison, it became evident that Random Forest (version 2) significantly outperformed the others, demonstrating both high accuracy and robustness. This finding highlights the algorithm's ability to capture complex patterns and relationships within the data, making it the most suitable choice for our specific task.

One of the key takeaways from this project is the crucial role of feature selection in improving model performance. The success of Random Forest (version 2) reinforces the idea that selecting the right features and dataset can dramatically impact an algorithm's predictive power. Conversely, the underperformance of other models suggests that either they were not well-suited for the given datasets or that additional optimization, such as hyperparameter tuning or alternative feature engineering strategies, may be necessary. These limitations serve as learning points for future work, where

further refinements could potentially enhance the performance of other algorithms as well.

Throughout this project, we faced challenges such as ensuring data quality, selecting appropriate evaluation metrics, and fine-tuning models for optimal performance. Addressing these challenges required iterative experimentation and analysis, ultimately strengthening our understanding of machine learning model selection and evaluation. Moving forward, future research could explore deeper hyperparameter optimization, ensemble learning techniques, and the integration of additional data sources to further improve predictive accuracy.

In summary, this study not only identified Random Forest as the most effective algorithm for our datasets but also reinforced the importance of feature selection and careful model evaluation. While there were challenges and limitations, they provided valuable lessons that can guide future work in predictive modelling. This research contributes to a deeper understanding of machine learning techniques and sets the stage for further advancements in data-driven decision-making.

6. AUTHORS CONTRIBUTIONS

Piotr Imiński: Conceptualization, Methodology, Software, Validation, Resources, Writing - Original Draft, Writing - Review & Editing, Project administration.

Jan Gorzela: Conceptualization, Methodology, Software, Formal analysis, Investigation, Writing - Original Draft.

Nadezhda Zhuravleva: Methodology, Software, Validation, Formal analysis, Resources, Writing - Original Draft.

7. REFERENCES

1. “<https://www.kaggle.com/datasets/warcoder/earthquake-dataset>.”
2. F. Waldhauser and D. P. Schaff, “Large-scale relocation of two decades of Northern California seismicity using cross-correlation and double-difference methods,” *J Geophys Res Solid Earth*, vol. 113, no. B8, Aug. 2008, doi: 10.1029/2007JB005479.
3. “<https://github.com/noobtdbs/Earthquake-Predictor/tree/main>.”
4. G. S. Manral and A. Chaudhary, “Prediction of Earthquake Using Machine Learning Algorithms,” in *2023 4th International Conference on Intelligent Engineering and Management (ICIEM)*, IEEE, May 2023, pp. 1–5. doi: 10.1109/ICIEM59379.2023.10166658.
5. M. Nurtas, Z. Zhantaev, and A. Altaibek, “Earthquake time-series forecast in Kazakhstan territory: Forecasting accuracy with SARIMAX,” *Procedia Comput Sci*, vol. 231, pp. 353–358, 2024, doi: 10.1016/j.procs.2023.12.216.
6. “https://www.javatpoint.com/earthquake-prediction-using-machine-learning?fbclid=IwY2xjawHm0SBleHRuA2FlbQlXMAABHV79IY7Y1qpJEO06V9SjIPNKBkDWJsMyvmzkXGni3-t2d6NKjegbeNHVcQ_aem_pVul_KHqL8Ucw7oDc5aMjg.”